

Systems for Design and Implementation: Assignment 2 Presentation

Project Name: FitTrack

Target Achieved: Bronze & Silver Challenges **MAX ACHIEVABLE SCORE**

1. Chosen Technology Stack & Justification

Pentru implementarea acestui nivel (Backend REST API + WebSockets), am decis sa extind aplicatia folosind un ecosistem tehnic unificat **JavaScript Fullstack**, dupa urmatorul criteriu de benchmarking:

- **Backend Framework: Node.js + Express.js**

Justificare: Spre deosebire de framework-uri greoaie bazate pe thread-uri concurente (ex: Spring Boot), arhitectura non-blocking I/O a Node.js este ideala pentru implementarea transmisiunilor *WebSockets* in timp real necesare provocarii de Silver. Permite un timp de dezvoltare ultra-rapid, folosind acelasi limbaj din Frontend (React).

- **Data Validation: Zod**

Justificare: In comparatie cu Joi sau express-validator, Zod ofera un mod declarativ mult mai curat si modern, ideal pentru maparea datelor.

- **Testing: Vitest & Supertest (cu v8 Coverage)**

Justificare: Vitest are suport nativ pentru module ES (ESM) folosite de mine pe frontend, fiind de cateva ori mai rapid decat vechiul Jest pe acest setup.

- **Real-time & Mocking: Socket.io + Faker.js**

Justificare: Faker.js a fost mentionat explicit ca standard in PDF-ul cerintei, iar Socket.io rezolva automat reincercarile de conectare (reconnect loop) folosite inteligent de mine pentru Modul Offline (cand pica serverul, datele se pastreaza in RAM local si se transmit ulterior).

2. Bronze Challenge: Implementation Proofs

- **NO PERSISTENCY (Data stored solely in RAM):**

Unde poti arata: Deschide `server/src/data/repository.js`.

Explicatie: Toate entitatile sunt retinute intr-o variabila interna **let workouts = []**. Nu exista conexiuni la Mongo sau Postgres. Redeschiderea serverului produce stergerea automata a tuturor datelor, fix cum cere documentatia.

- **Server-side and Client-side Validation for ANY input:**

Client-side: Da click pe adaugarea/editarea unui formular pe Frontend cu intrari gresite sau numar negativ (React arunca avertizari vizuale roșii imediat).

Server-side: In `routes/workouts.js`, la rutele de POST/PUT, se gasesc scheme validatoare **Zod**. Un request cu campuri gresite (fortat prin Postman) va fi respins garantat server-side prin cod HTTPS 400 Bad Request.

- **Unit Tests & Code Coverage for CRUD logic:**

Ce sa rulezi: Executa comanda `npm run test:coverage` in terminalul /server.

Explicatie: Va fi expus un scor nativ in consola de **peste 95% Code Coverage** pe toata logica CRUD, demonstrand ca totul e testat riguros (pentru status 200, 400 bad payload si 404 not found).

- **Server-side Pagination:**

Unde poti arata: Apasa Next/Prev pe componenta React Table-ul curent. Network tab va arata requesturi ca `GET /workouts?page=1&limit=10`.

3. Silver Challenge: Implementation Proofs

- **Offline Support & Local Data Synchronization:**

Cum demonstrezi practic: Taie internetul / Opreste Terminalul in timp ce aplicatia curge. Editezi sau stegi rânduri (Offline mode vizibil pe UI).

Explicatie tehnica: Am implementat o coada ascunsa (Queue) in **LocalStorage** si pe un Listener. La repornirea Serverului de Node, React-ul simte reconectarea WebSocketului, preia cererile picate asincron si da Flush si se sincronizeaza silentios catre baza de date a serverului din Node.

- **Asynchronous Simulation Loop (Faker) + WebSocket Reactivity (Master/Detail + Charts):**

Cum demonstrezi: Mergi in sectiunea `Bazinga Dashboard` si apasa `Start Simulation`.

Explicatie tehnica: Apasarea cere executia rutei de server **POST /simulation/start**.

Acest controller din Node activeaza bucla **setInterval**. O data la scurt timp extrage si emite **@faker-js/faker** complet valid prin **io.emit('new_workouts')** pe conducta WebSockets. UI-ul asculta si introduce totul in **useState()** modificand live tabelul Master/Detail si adaptand graficele radiale (SVG Animation Charts).

Document prepared via FitTrack Repository Codebase Verification.

4. Gold Challenge: Implementation Proofs

BE AMBITIOUS - GraphQL backend layer:

Ce poti arata: Porneste serverul si deschide `http://localhost:3000/graphql`. In consola GraphQL se pot testa query-uri si mutations live.

Explicatie: Pe langa REST API, aceeasi logica de backend este expusa si prin GraphQL. Frontend-ul poate cere exact campurile necesare pentru workouts si exercises, fara sa primeasca date inutile.

Exemple: `getWorkouts`, `addWorkout`, `updateWorkout`, `deleteWorkout`.

BE RESPONSIVE - Infinite Scroll on frontend:

Ce poti arata: In Dashboard, deruleaza zona cu workouts. Cand ajungi aproape de final, urmatorul set de rezultate se incarca automat.

Explicatie: In loc de paginare manuala cu Next/Prev, aplicatia foloseste infinite scroll. Frontend-ul cere urmatoarea pagina doar cand este nevoie, folosind parametri precum `offset` si `limit`. Astfel se reduce traficul si interactiunea este mai fluida.

BE DILIGENT - 1-to-many relation fullstack:

Model: un **Workout** poate contine mai multe **Exercises**. Workout-ul este entitatea parinte, iar exercitiile sunt elementele copil.

Ce poti arata: Deschide pagina de detalii a unui workout, foloseste sectiunea Exercises si adauga un exercitiu nou. Apoi sterge un exercitiu existent pentru a demonstra operatiile CRUD pe relatia 1-N.

Explicatie: Relatia este acoperita din frontend pana in backend: datele sunt validate, trimise la server si actualizate in interfata fara refresh manual.

Demo scurt pentru prezentare:

1. Deschid `/graphql` si rulez o cerere simpla.
2. In Dashboard fac scroll si arat incarcarea automata a datelor.
3. In pagina de detalii adaug/sterg un exercise pentru un workout.

Concluzie: Gold-ul extinde proiectul cu GraphQL, infinite scroll si o relatie reala 1-N intre entitati, fara sa schimbe scopul principal al aplicatiei.